Код класса AuthManager:

```
import sqlite3

class AuthManager:
    def __init__(self, connection):
        self.connection = connection
        self.create_tables()

    def create_tables(self):
        with self.connection:
            self.connection.execute("""
                CREATE TABLE IF NOT EXISTS users (
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
                    username TEXT NOT NULL UNIQUE,
                    password TEXT NOT NULL,
                    country TEXT NOT NULL,
                    balance REAL NOT NULL
                )
            """)

    def register_user(self, username, password, country, balance):
        with self.connection:
            self.connection.execute("""
                INSERT INTO users (username, password, country, balance)
                VALUES ('{}', '{}', '{}', {})
            """.format(username, password, country, balance))

    def authenticate_user(self, username, password):
        cursor = self.connection.cursor()
        cursor.execute("""
            SELECT * FROM users
            WHERE username = '{}' AND password = '{}'
        """.format(username, password))
        return cursor.fetchone()

    def delete_user(self, user_id):
        with self.connection:
            self.connection.execute("""
                DELETE FROM users WHERE id = {}
            """.format(user_id))
```

```
    def get_user_by_id(self, user_id):
        cursor = self.connection.cursor()
        cursor.execute("""
            SELECT * FROM users WHERE id = {}
        """.format(user_id))
        return cursor.fetchone()

    def count_users_by_country(self, country):
        cursor = self.connection.cursor()
        cursor.execute("""
            SELECT COUNT(*) FROM users WHERE country = '{}'
        """.format(country))
        return cursor.fetchone()[0]

    def transfer_balance(self, from_user_id, to_user_id, amount):
        with self.connection:
            # Проверяем, достаточно ли средств
            cursor = self.connection.cursor()
            cursor.execute("SELECT balance FROM users WHERE id =
{}".format(from_user_id))
            from_balance = cursor.fetchone()[0]

            if from_balance < amount:
                raise ValueError("Insufficient funds")

            # Выполняем перевод
            self.connection.execute("""
                UPDATE users SET balance = balance - {} WHERE id = {}
            """.format(amount, from_user_id))
            self.connection.execute("""
                UPDATE users SET balance = balance + {} WHERE id = {}
            """.format(amount, to_user_id))
```

Код тестов:

```
%%writefile test_auth.py
import pytest
import sqlite3
import sys

class AuthManager:
    def __init__(self, connection):
        self.connection = connection
        self.create_tables()

    def create_tables(self):
        with self.connection:
            self.connection.execute("""
                CREATE TABLE IF NOT EXISTS users (
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
                    username TEXT NOT NULL UNIQUE,
                    password TEXT NOT NULL,
                    country TEXT NOT NULL,
                    balance REAL NOT NULL
                )
            """)
```

```
    def register_user(self, username, password, country, balance):
        with self.connection:
            self.connection.execute("""
                INSERT INTO users (username, password, country, balance)
                VALUES ('{}', '{}', '{}', '{}')
            """.format(username, password, country, balance))

    def authenticate_user(self, username, password):
        cursor = self.connection.cursor()
        cursor.execute("""
            SELECT * FROM users
            WHERE username = '{}' AND password = '{}'
        """.format(username, password))
        return cursor.fetchone()

    def delete_user(self, user_id):
        with self.connection:
            self.connection.execute("""
                DELETE FROM users WHERE id = {}
            """.format(user_id))

    def get_user_by_id(self, user_id):
        cursor = self.connection.cursor()
        cursor.execute("""
            SELECT * FROM users WHERE id = {}
        """.format(user_id))
        return cursor.fetchone()

    def count_users_by_country(self, country):
        cursor = self.connection.cursor()
        cursor.execute("""
            SELECT COUNT(*) FROM users WHERE country = '{}'
        """.format(country))
        return cursor.fetchone()[0]

    def transfer_balance(self, from_user_id, to_user_id, amount):
        with self.connection:
            cursor = self.connection.cursor()
            cursor.execute("SELECT balance FROM users WHERE id =
{}".format(from_user_id))
            from_balance = cursor.fetchone()[0]
            if from_balance < amount:
                raise ValueError("Insufficient funds")
            self.connection.execute("""
                UPDATE users SET balance = balance - {} WHERE id = {}
            """.format(amount, from_user_id))
            self.connection.execute("""
                UPDATE users SET balance = balance + {} WHERE id = {}
            """.format(amount, to_user_id))
```

```python
@pytest.fixture
def db():
    """Фикстура: временная SQLite в памяти."""
    connection = sqlite3.connect(":memory:")
    yield connection
    connection.close()


@pytest.fixture
def auth_manager(db):
    """Фикстура: AuthManager с подключённой БД."""
    return AuthManager(db)



@pytest.mark.basic
def test_register_user(auth_manager):
    """Базовый тест: регистрация пользователя."""
    auth_manager.register_user("ivan", "pass123", "Russia", 1000)
    user = auth_manager.get_user_by_id(1)
    assert user is not None
    assert user[1] == "ivan"
    assert user[3] == "Russia"


@pytest.mark.basic
def test_authenticate_user(auth_manager):
    """Базовый тест: аутентификация пользователя."""
    auth_manager.register_user("petr", "secret", "Russia", 500)
    user = auth_manager.authenticate_user("petr", "secret")
    assert user is not None
    assert user[1] == "petr"


@pytest.mark.basic
def test_delete_user(auth_manager):
    """Базовый тест: удаление пользователя."""
    auth_manager.register_user("anna", "qwerty", "Germany", 200)
    auth_manager.delete_user(1)
    user = auth_manager.get_user_by_id(1)
    assert user is None



@pytest.mark.parametrize("country, expected_count", [
    ("Russia", 2),
    ("Germany", 1),
    ("France", 0)
])
def test_count_users_by_country(auth_manager, country, expected_count):
    """Параметризованный тест: подсчёт пользователей по странам."""
```

```
    auth_manager.register_user("user1", "pass", "Russia", 100)
    auth_manager.register_user("user2", "pass", "Russia", 200)
    auth_manager.register_user("user3", "pass", "Germany", 300)
    count = auth_manager.count_users_by_country(country)
    assert count == expected_count


@pytest.mark.parametrize("username, password, expected", [
    ("ivan", "pass123", True),
    ("ivan", "wrong", False),
    ("nobody", "pass123", False)
])
def test_authenticate_parametrized(auth_manager, username, password,
expected):
    """Параметризованный тест: аутентификация с разными данными."""
    auth_manager.register_user("ivan", "pass123", "Russia", 1000)
    result = auth_manager.authenticate_user(username, password)
    assert (result is not None) == expected


@pytest.mark.parametrize("amount, expected_balance", [
    (100, 900),
    (500, 500),
    (900, 100)
])
def test_transfer_balance_parametrized(auth_manager, amount,
expected_balance):
    """Параметризованный тест: перевод средств."""
    auth_manager.register_user("sender", "pass", "Russia", 1000)
    auth_manager.register_user("receiver", "pass", "Russia", 0)
    auth_manager.transfer_balance(1, 2, amount)
    sender = auth_manager.get_user_by_id(1)
    assert sender[4] == expected_balance



@pytest.mark.exception
def test_transfer_insufficient_funds(auth_manager):
    """Тест исключения: недостаточно средств."""
    auth_manager.register_user("poor", "pass", "Russia", 50)
    auth_manager.register_user("rich", "pass", "Russia", 1000)
    with pytest.raises(ValueError, match="Insufficient funds"):
        auth_manager.transfer_balance(1, 2, 100)


@pytest.mark.exception
def test_get_nonexistent_user(auth_manager):
    """Тест исключения: обращение к несуществующему пользователю."""
    with pytest.raises(TypeError):
        auth_manager.transfer_balance(999, 1, 100)
```

```python
@pytest.mark.security
def test_sql_injection(auth_manager):
    """Тест на SQL-инъекцию: попытка обойти аутентификацию."""
    auth_manager.register_user("admin", "secret", "Russia", 10000)

    malicious_password = "' OR '1'='1"
    result = auth_manager.authenticate_user("admin", malicious_password)


    assert result is None, f"КРИТИЧЕСКАЯ УЯЗВИМОСТЬ: SQL-инъекция удалась!
Получены данные: {result}"


@pytest.mark.skipif(sys.version_info < (3, 7), reason="Требуется Python
3.7+")
def test_python_version():
    """Пропускаемый тест: проверка версии Python."""
    assert True


@pytest.mark.skip(reason="Тест временно отключён")
def test_skipped_example():
    """Пример пропускаемого теста."""
    assert False
```

```
!pytest -v --maxfail=2 --tb=short test_auth.py
```

```
============================= test session starts ==============================
platform linux -- Python 3.13.15, pytest-8.4.2, pluggy-1.6.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /content
configfile: pytest.ini
plugins: langsmith-0.12.1, anyio-4.14.2, typeguard-4.6.0
collected 17 items

test_auth.py::test_register_user PASSED                                  [  5%]
test_auth.py::test_authenticate_user PASSED                              [ 11%]
test_auth.py::test_delete_user PASSED                                    [ 17%]
test_auth.py::test_count_users_by_country[Russia-2] PASSED               [ 23%]
test_auth.py::test_count_users_by_country[Germany-1] PASSED              [ 29%]
test_auth.py::test_count_users_by_country[France-0] PASSED               [ 35%]
test_auth.py::test_authenticate_parametrized[ivan-pass123-True] PASSED   [ 41%]
test_auth.py::test_authenticate_parametrized[ivan-wrong-False] PASSED    [ 47%]
test_auth.py::test_authenticate_parametrized[nobody-pass123-False] PASSED [ 52%]
test_auth.py::test_transfer_balance_parametrized[100-900] PASSED         [ 58%]
test_auth.py::test_transfer_balance_parametrized[500-500] PASSED         [ 64%]
test_auth.py::test_transfer_balance_parametrized[900-100] PASSED         [ 70%]
test_auth.py::test_transfer_insufficient_funds PASSED                    [ 76%]
test_auth.py::test_get_nonexistent_user PASSED                           [ 82%]
test_auth.py::test_sql_injection PASSED                                  [ 88%]
test_auth.py::test_python_version PASSED                                 [ 94%]
test_auth.py::test_skipped_example SKIPPED (Тест временно отключён)      [100%]

======================== 16 passed, 1 skipped in 0.05s =========================
```

Рисунок 1 – Успешный запуск теста

```python
@pytest.mark.parametrize("country, expected_count", [
    ("Russia", 5),
    ("Germany", 1),
    ("France", 0)
])
```

Рисунок 2 – Делаем ошибку

```
!pytest -v --maxfail=2 --tb=short test_auth.py

============================= test session starts ==============================
platform linux -- Python 3.13.15, pytest-8.4.2, pluggy-1.6.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /content
configfile: pytest.ini
plugins: langsmith-0.12.1, anyio-4.14.2, typeguard-4.6.0
collected 17 items

test_auth.py::test_register_user PASSED                                  [  5%]
test_auth.py::test_authenticate_user PASSED                              [ 11%]
test_auth.py::test_delete_user PASSED                                    [ 17%]
test_auth.py::test_count_users_by_country[Russia-5] FAILED               [ 23%]
test_auth.py::test_count_users_by_country[Germany-1] PASSED              [ 29%]
test_auth.py::test_count_users_by_country[France-0] PASSED               [ 35%]
test_auth.py::test_authenticate_parametrized[ivan-pass123-True] PASSED   [ 41%]
test_auth.py::test_authenticate_parametrized[ivan-wrong-False] PASSED    [ 47%]
test_auth.py::test_authenticate_parametrized[nobody-pass123-False] PASSED [ 52%]
test_auth.py::test_transfer_balance_parametrized[100-900] PASSED         [ 58%]
test_auth.py::test_transfer_balance_parametrized[500-500] PASSED         [ 64%]
test_auth.py::test_transfer_balance_parametrized[900-100] PASSED         [ 70%]
test_auth.py::test_transfer_insufficient_funds PASSED                    [ 76%]
test_auth.py::test_get_nonexistent_user PASSED                           [ 82%]
test_auth.py::test_sql_injection PASSED                                  [ 88%]
test_auth.py::test_python_version PASSED                                 [ 94%]
test_auth.py::test_skipped_example SKIPPED (Тест временно отключён)      [100%]

=================================== FAILURES ===================================
_____________________ test_count_users_by_country[Russia-5] _____________________
test_auth.py:124: in test_count_users_by_country
    assert count == expected_count
E   assert 2 == 5
=========================== short test summary info ============================
FAILED test_auth.py::test_count_users_by_country[Russia-5] - assert 2 == 5
=================== 1 failed, 15 passed, 1 skipped in 0.13s ====================
```

Рисунок 3 – Запуск теста с ошибкой

```
!pytest -v test_auth.py::test_sql_injection

============================= test session starts ==============================
platform linux -- Python 3.13.16, pytest-8.4.2, pluggy-1.6.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /content
configfile: pytest.ini
plugins: anyio-4.15.1, platformdirs-4.12.2, langsmith-0.14.2, typeguard-4.6.0
collected 1 item

test_auth.py::test_sql_injection FAILED                                  [100%]

=================================== FAILURES ===================================
_______________________________ test_sql_injection _______________________________

auth_manager = <test_auth.AuthManager object at 0x78aadc806e40>

    @pytest.mark.security
    def test_sql_injection(auth_manager):
        """Тест на SQL-инъекцию: попытка обойти аутентификацию."""
        auth_manager.register_user("admin", "secret", "Russia", 10000)

        malicious_password = "' OR '1'='1"
        result = auth_manager.authenticate_user("admin", malicious_password)

>       assert result is None, f"КРИТИЧЕСКАЯ УЯЗВИМОСТЬ: SQL-инъекция удалась! Получены данные: {result}"
E       AssertionError: КРИТИЧЕСКАЯ УЯЗВИМОСТЬ: SQL-инъекция удалась! Получены данные: (1, 'admin', 'secret', 'Russia', 10000.0)
E       assert (1, 'admin', 'secret', 'Russia', 10000.0) is None

test_auth.py:184: AssertionError
=========================== short test summary info ============================
FAILED test_auth.py::test_sql_injection - AssertionError: КРИТИЧЕСКАЯ УЯЗВИМОСТЬ: SQL-инъекция удалась! Получены данн...
============================== 1 failed in 0.10s ===============================
```

Рисунок 4 – Тест на SQL-инъекцию

Пояснение применённых техник:

Базовые тесты:

test_register_user - регистрация пользователя

test_authenticate_user - аутентификация

test_delete_user - удаление пользователя

Параметризованные тесты:

test_count_users_by_country - подсчёт по странам (Russia, Germany, France)

test_authenticate_parametrized - аутентификация с разными данными

test_transfer_balance_parametrized - перевод средств с разными суммами

Тесты исключений:

test_transfer_insufficient_funds - недостаточно средств

test_get_nonexistent_user - обращение к несуществующему пользователю

Фикстуры:

db - временная SQLite в памяти

auth_manager - экземпляр AuthManager

Метки:

@pytest.mark.basic

@pytest.mark.exception

@pytest.mark.security

Пропускаемый тест:

test_skipped_example - SKIPPED

Тест на SQL-инъекции:

test_sql_injection — проверка уязвимости